

Consultorios Médicos — Lector de Excel

Documentación técnica del proyecto

Versión: Angular 18 · SPA cliente · Sin backend de aplicación

Parte I — Visión general y arquitectura

1. Qué es la aplicación

Aplicación web **SPA (Single Page Application)** en **Angular 18** que permite cargar archivos Excel (`.xlsx` , `.xls`) con plantillas del dominio de consultorios médicos (MoCE), extraer datos de hojas específicas, aplicar reglas de negocio en el navegador y mostrarlos en tablas interactivas.

No existe backend de aplicación en el sentido habitual: no hay API REST, base de datos ni autenticación. El archivo Excel **no se sube a ningún servidor**; se procesa íntegramente en el cliente con la API `FileReader` y la librería **SheetJS (xlsx)**.

En **producción (Docker)**, el único “servidor” es **Nginx**, que entrega archivos estáticos generados por `ng build` .

2. Arquitectura por capas

Capa	Tecnología	Responsabilidad
Presentación	Angular standalone + HTML/CSS	UI, rutas, tablas, modal
Estado	<code>ExcelStateService</code>	Compartir datos entre rutas en memoria
Lógica de negocio	<code>ExcelReaderService</code>	Parseo Excel, mapeo de columnas, reglas MoCE
Persistencia	Ninguna	Los datos se pierden al recargar la página
Despliegue	<code>ng build</code> + Nginx	Servir bundle estático

Flujo de alto nivel

```
Usuario → Archivo .xlsx
  → UploadZone (valida extensión, emite File)
  → HomePage.onFileSelected()
  → ExcelReaderService.readFile() [FileReader + SheetJS]
  → ExcelStateService.data
  → DataTable / Modal (vistas)
```

3. Backend: qué hay y qué no hay

Existe	No existe
Nginx sirviendo HTML/JS/CSS en Docker	API REST / GraphQL
Reglas de negocio en <code>ExcelReaderService</code> (cliente)	Subida de archivos al servidor

Estado en memoria (ExcelStateService)

Base de datos, sesiones, login

La lógica que en otros sistemas viviría en un servidor (cruzar catálogo, generar códigos de consultorio, validar horarios) está en `ExcelReaderService`, ejecutándose en el navegador del usuario.

4. Estructura del proyecto

```
src/app/
├─ app.component.ts      # Shell: navegación global
├─ app.config.ts         # provideRouter
├─ app.routes.ts         # Rutas
├─ models/excel.models.ts # Interfaces TypeScript
├─ services/
│   ├─ excel-reader.service.ts # Lectura y transformación
│   └─ excel-state.service.ts  # Estado global en memoria
├─ pages/
│   ├─ home/              # Carga Excel + tabla procesada
│   └─ personal-operativo/ # Tabla datos crudos
└─ components/
    ├─ upload-zone/       # Carga de archivo
    ├─ data-table/        # Tabla genérica
    └─ modal/             # Detalle de fila
```

5. Rutas

Ruta	Componente	Función
/	HomePage	Subir Excel; mostrar Alta Usuarios — Información procesada
/personal-operativo	PersonalOperativoPage	Misma data en memoria; tabla Personal Operativo (25 columnas)
**	redirect → /	

6. Modelos de datos (excel.models.ts)

- **PersonalOperativo** : fila mapeada desde hoja "PERSONAL OPERATIVO" (entidad, CLUES, especialidad, horarios, días L–D, etc.).
- **PersonaProcesada** : salida transformada (consultorio generado, horarios formateados, turno, días de consulta, flag `revisado`).
- **Catalogo** : `descripcion` + `nomenclatura` desde hoja "Catálogo".
- **SheetData** : `{ personalOperativo, personasProcesadas, allSheets, rawHeaders }` .

7. Servicios

ExcelStateService

- Singleton `providedIn: 'root'` .
- Propiedad `data: SheetData | null` .
- Comparte el resultado entre `/` y `/personal-operativo` sin recargar el archivo.

- No usa RxJS ni Signals; estado mutable simple.

ExcelReaderService — reglas de negocio

`readFile(file)` : `FileReader.readAsArrayBuffer` → `XLSX.read` → `extractData(wb)` .

`extractData` (resumen):

1. Lista hojas del workbook y cabeceras por hoja.
2. Hoja cuyo nombre contiene "personal operativo" → filas JSON.
3. Mapeo flexible de columnas (`findColumnKey` , normalización de acentos).
4. Construye `personalOperativo[]` (filas con nombre).
5. Hoja "catalogo" → array `catalogos[]` .
6. Por cada fila genera `PersonaProcesada` :

Regla	Comportamiento
Nombre completo	<code>apellidoPaterno + apellidoMaterno + nombre</code>
Nomenclatura	Similitud de texto vs catálogo; umbral 0.5; si falla → <code>SIN_CATALOGO(especialidad)</code>
Código consultorio	Consecutivo por base de nomenclatura y médico
Horas	Decimales Excel → HH:mm
Horario citas	Valida múltiplo del intervalo; advertencia si no cuadra
Turno	MATUTINO / VESPERTINO / JORNADA COMPLEMENTARIA
Días	Celdas con X → L-M-MI-J-V-S-D
Constantes	<code>subRol: MEDICO ESPECIALISTA</code> , <code>tipoVisita: CONSULTORIO</code>

8. Páginas y componentes (resumen)

Pieza	Rol
HomePage	Orquesta carga, guarda en state, tabla procesada, ordenar, modal
PersonalOperativoPage	Lee <code>state.data</code> ; tabla cruda; sin upload
UploadZoneComponent	Input file + drag & drop; emite <code>File</code>
DataTableComponent	Tabla genérica con <code>@Input rows/columns</code> , <code>@Output rowClick</code>
ModalComponent	Detalle de fila; marcar revisado en memoria

9. Sistema de estilos

- **CSS plano** por componente; sin Bootstrap/Tailwind/SCSS.
- Global mínimo en `src/styles.css` (Segoe UI , fondo `#f8f7f4`).
- Paleta principal: verde `#1a4d3a` , fondos `#eef7f2` , alertas amarillo `#fffbea` .
- Tablas: cabecera sticky, scroll max 380px.
- Iconos: SVG inline en plantillas.

10. Stack y despliegue

Pieza	Uso
Angular 18	Standalone components, Zone.js
TypeScript 5.4	Tipado
SheetJS (xlsx)	Lectura binaria del Excel
Dev	npm start → http://localhost:4200
Prod	npm run build → dist/consultorios-medicos/browser
Docker	Node 20 build + Nginx :80 → mapeo local :8080

11. Limitaciones actuales

- Sin persistencia ni exportación del resultado.
- `revisado` solo en memoria (mutación del objeto fila).
- Duplicación de definición `columnsPersonal` en dos páginas.
- `alertasSinCatalogo` calculado en Home pero no mostrado en HTML.
- Validación de archivo solo por extensión en UploadZone.

Parte II — Flujo técnico componente por componente

En esta app, “capa de lógica” = `ExcelReaderService` en el navegador. No hay servidor que reciba el archivo.

Cadena completa al subir un Excel

```
HTML (input / drag)
→ UploadZoneComponent
→ @Output fileSelected
→ HomePage.onFileSelected()
→ ExcelReaderService.readFile()
→ ExcelStateService.data
→ DataTableComponent
```

1. UploadZoneComponent — cargar el Excel

HTML

```
<input #fileInput type="file" accept=".xlsx,.xls" style="display:none"
  (change)="onFileChange($event)">
```

Elemento	Función
#fileInput	Referencia al input nativo del DOM

accept	Filtro en diálogo del SO (no validación estricta)
display:none	UI visible es el div; el input está oculto
(change)	Dispara onChange al elegir archivo

Clic en zona visible:

```
<div (click)="fileInput.click()"> <!-- abre diálogo del SO -->
```

Drag & drop en contenedor:

```
(dragover)="onDragOver($event)"
(dragleave)="onDragLeave()"
(drop)="onDrop($event)"
```

TypeScript

onChange(e: Event)

1. e.target = el <input> .
2. files[0] = objeto File del navegador (binario en RAM).
3. Llama emitFile(file) .

onDrop(e: DragEvent)

1. preventDefault() — evita abrir el archivo en otra pestaña.
2. e.dataTransfer.files[0] → emitFile .

emitFile(file)

1. Regex \.(xlsx|xls)\$ — si no coincide, **return silencioso** (sin mensaje UI).
2. this.fileName = file.name — actualiza plantilla.
3. this.fileSelected.emit(file) — **EventEmitter** hacia el padre.

UploadZone no lee Excel. Solo valida extensión y reenvía el File .

2. HomePage — enlace padre → servicio

HTML

```
<app-upload-zone (fileSelected)="onFileSelected($event)" />
```

- (fileSelected) = escucha @Output del hijo.
- \$event = el File emitido.

onFileSelected(file: File)

Paso	Código / efecto
1	errorMsg = '', state.data = null — limpia estado previo

2	<code>await excelService.readFile(file)</code>
3	<code>state.data = resultado</code>
4	Filtra SIN_CATALOGO → <code>alertasSinCatalogo[]</code>
5	<code>catch</code> → <code>errorMsg = err.message</code> → <code>*ngIf</code> en error-box

La página **no parsea** el Excel; delega al servicio.

3. ExcelReaderService — lógica en el cliente

`readFile(file: File): Promise<SheetData>`

Paso	API	Efecto
1	<code>new FileReader()</code>	API navegador
2	<code>readAsArrayBuffer(file)</code>	Lee bytes localmente
3	<code>onload</code> → <code>XLSX.read(result, {type: 'array'})</code>	Workbook SheetJS
4	<code>extractData(wb)</code>	Objetos tipados
5	<code>resolve / reject</code>	Promise para <code>await</code> en HomePage

El archivo nunca sale del equipo del usuario.

`extractData(wb)` — pipeline

SheetNames

```

→ rawHeaders por hoja
→ hoja "personal operativo" → sheetToRows → { "Col Excel": valor }
→ findColumnKey → keys.apellidoPaterno, etc.
→ personalOperativo[]
→ hoja "catalogo" → catalogos[]
→ loop filas → personasProcesadas[]
→ return SheetData

```

`findColumnKey` : normaliza acentos (NFD), compara alias ("apellido paterno" , "apellido paterno"), devuelve **nombre real** del encabezado en el Excel para indexar `r[keys.xxx]` .

4. ExcelStateService

```
data: SheetData | null = null;
```

- `HomePage` escribe tras cargar.
- `PersonalOperativoPage` lee sin nuevo upload.
- Angular repinta por `*ngIf="state.data"` en plantillas.

5. HomePage → DataTableComponent

```
<div *ngIf="state.data">
  <app-data-table
    [rows]="state.data.personasProcesadas"
    [columns]="columnsProcesadas"
    (rowClick)="selectedRow = $event" />
</div>
```

Binding	Tipo	Significado
[rows]	@Input	Array a renderizar
[columns]	@Input	{ label, key }[]
(rowClick)	@Output	Fila clicada → modal

columnsProcesadas define que `key: 'consultorio'` muestra `row.consultorio`.

6. DataTableComponent

Inputs / Outputs

```
@Input() rows: any[] = [];  
@Input() columns: { label: string; key: string }[] = [];  
@Output() rowClick = new EventEmitter<any>();
```

Template (lógica de render)

```
<tr *ngFor="let row of rows" (click)="rowClick.emit(row)">  
  <td *ngFor="let col of columns">  
    {{ row[col.key] }} <!-- acceso dinámico -->  
  </td>  
</tr>
```

- `row[col.key]` → ej. `row['consultorio']`.
- Si valor incluye `SIN_CATALOGO` → clase `cell-alert` y slice del texto.
- `[class.revisado]="row.revisado"` — fila marcada en modal.

Sin lógica de Excel — solo presentación.

7. ModalComponent

```
<app-modal *ngIf="selectedRow">  
  [row]="selectedRow"  
  [columns]="columnsProcesadas">
```

```
(cerrar)="selectedRow = null"
(toggleRevisado)="selectedRow.revisado = !selectedRow.revisado" />
```

Evento	Efecto
overlay click	cerrar.emit()
modal click	stopPropagation() — no cierra
toggleRevisado	Muta mismo objeto en state.data.personasProcesadas

Cuerpo: `*ngFor="let col of columns" → {{ row[col.key] }}` .

8. PersonalOperativoPage

```
<app-data-table *ngIf="state.data"
  [rows]="state.data.personalOperativo"
  [columns]="columnsPersonal" />
```

Mismo `DataTable` , otro array (datos crudos, 25 columnas). Sin `UploadZone` ; requiere carga previa en `/` .

9. Ordenar (HomePage)

```
<button (click)="ordenarAlfabeticamente()">
```

```
state.data.personasProcesadas.sort((a,b) =>
  a.consultorio.localeCompare(b.consultorio));
```

Orden **in place**; misma referencia de array → tabla se actualiza.

10. Glosario Angular (este proyecto)

HTML	TypeScript	Rol
(click)="fn()"	método	Evento DOM
(change)="onFileChange(\$event)"	Event	Cambio en input file
(fileSelected)="handler(\$event)"	@Output hijo	Hijo → padre
[rows]="array"	@Input	Padre → hijo
*ngIf="cond"	—	Render condicional
*ngFor="let x of arr"	—	Lista en template
#fileInput	referencia plantilla	fileInput.click()

11. Secuencia resumida (subida de archivo)

1. Usuario elige `.xlsx` → `onFileChange` o `onDrop` .
2. `emitFile` → `fileSelected.emit(file)` .
3. `HomePage.onFileSelected(file)` .
4. `ExcelReaderService.readFile` → `FileReader` + `XLSX` + `extractData`.
5. `ExcelStateService.data` = `SheetData` .
6. `DataTable` recibe `personasProcesadas` .
7. Click fila → `selectedRow` → `Modal` .

Resumen final

Componente	Responsabilidad
UploadZone	Entregar file (extensión .xlsx/.xls)
HomePage	Orquestar lectura y estado
ExcelReaderService	Parseo + reglas de negocio
ExcelStateService	Memoria compartida entre rutas
DataTable	Render tabular genérico
Modal	Detalle y flag revisado
PersonalOperativoPage	Vista alternativa mismos datos

Producción: Nginx solo sirve estáticos; no procesa Excel.
